CSA4708 – Deep Learning: Assignment 2 (CO2) Solutions

Saveetha School of Engineering, SIMATS

Name: E.Kavya

Reg. No.: 192424365

Scenario: Smart Agriculture and Crop Disease Detection

Task 1 – Deep Feed-Forward Network Architecture (5 Marks)

a) Architecture

Layer	Neurons	Role
Input	4	rainfall, temperature, soil moisture, fertilizer quantity
Hidden 1	8	learns combinations of the raw features (ReLU)
Hidden 2	4	learns higher-level combinations (ReLU)
Output	1	predicted crop yield (linear activation, since this is regression)

Structure: $4 \rightarrow 8 \rightarrow 4 \rightarrow 1$, fully connected.

b) Dimensions (column-vector convention, $z = Wa + b$)

Parameter	Shape	Count
W^1	8×4	32
b^1	8×1	8
W^2	4×8	32
b^2	4×1	4
W^3	1×4	4
b^3	1×1	1

Total trainable parameters = 32 + 8 + 32 + 4 + 4 + 1 = **81**.

c) Forward propagation

Let $x = [\text{rainfall, temperature, soil moisture, fertilizer}]^T$ (4×1).

1. Hidden layer 1: $z^1 = W^1x + b^1$ (8×1), $a^1 = \text{ReLU}(z^1)$
2. Hidden layer 2: $z^2 = W^2a^1 + b^2$ (4×1), $a^2 = \text{ReLU}(z^2)$
3. Output: $\hat{y} = W^3a^2 + b^3$ $(1\times1$, linear$)$

Combined: $\hat{y} = W^3 \cdot \text{ReLU}(W^2 \cdot \text{ReLU}(W^1x + b^1) + b^2) + b^3$.

Each layer takes the previous layer's output, computes a weighted sum plus bias, and applies a non-linear activation. The final scalar $\hat{y}$ is the predicted yield. Inputs should be normalized beforehand.

Task 2 – Forward Propagation and Activation Functions (2 Marks)

Inputs $x = (0.6, 0.8, 0.5)$, weights $w = (0.4, 0.3, 0.2)$, bias $b = 0.1$.

a) Weighted sum

$z = (0.6\times0.4) + (0.8\times0.3) + (0.5\times0.2) + 0.1 = 0.24 + 0.24 + 0.10 + 0.1 =$ **0.68**

b) ReLU

$\text{ReLU}(z) = \max(0, 0.68) =$ **0.68**

Interpretation: z is positive, so the neuron is active and passes its value unchanged to the next layer. Had z been negative, the output would be 0 (neuron inactive).

Task 3 – Gradient-Based Learning and Loss Calculation (6 Marks)

a) Need for a loss function and choice

A loss function converts the gap between prediction and truth into a single number. Without it the network has no measure of how wrong it is and no quantity to minimize or differentiate. For regression (yield prediction), a suitable choice is **Mean Squared Error (MSE)**. It is smooth and differentiable, and it penalizes large errors more strongly. (Huber loss is a robust alternative if outliers exist.)

b) Squared error

$y = 50$, $\hat{y} = 42 \rightarrow L = (50 - 42)^2 = 8^2 =$ **64**

c) Interpretation

The loss of 64 is large relative to a perfect value of 0. The model under-predicts by 8 units, so its parameters need adjustment. Because the error is squared, the loss grows quickly as the gap grows.

d) Gradient-based learning

1. Forward pass gives $\hat{y}$ and the loss L.
2. Compute the gradient $\partial L/\partial \theta$ for every weight and bias. It shows how L changes if that parameter changes.
3. Update each parameter in the direction that reduces loss: $\theta \leftarrow \theta - \eta\, \partial L/\partial \theta$.
4. Repeat over many iterations/epochs until the loss converges.

Here $\partial L/\partial \hat{y} = -2(y - \hat{y}) = -16$, which is propagated backward to all parameters.

Task 4 – Gradient Descent and Weight Update (4 Marks)

a) Update

$w_new = w - \eta \cdot \partial L/\partial w = 0.5 - (0.1 \times 0.8) = 0.5 - 0.08 =$ **0.42**

b) Why the opposite direction?

The gradient points in the direction of steepest increase of the loss. We want to decrease the loss, so we move against it. A positive gradient (0.8) means increasing w raises the loss, so w is reduced.

c) Learning rate effects

- Extremely small η: very slow convergence, many iterations needed, may stall in flat regions or plateaus, high training cost.
- Extremely large η: steps overshoot the minimum, loss oscillates or diverges, training becomes unstable.
- A well-chosen η (often with a decay schedule) gives fast yet stable convergence.

Task 5 – Back-Propagation Through Multiple Layers (3 Marks)

a) Contribution of weights to error

After the forward pass computes $\hat{y}$ and loss L, backpropagation moves from the output layer back to the input layer. At each layer it computes the error signal δ (how much that layer's pre-activation contributed to the loss) and from it the gradient for that layer's weights. Weights that influenced the output more receive larger gradients, so each weight's share of the error is quantified.

b) Role of the chain rule

The loss depends on early weights through a chain of functions, so gradients are computed by multiplying local derivatives:

$\partial L/\partial W^1 = \partial L/\partial \hat{y} \cdot \partial \hat{y}/\partial a^2 \cdot \partial a^2/\partial z^2 \cdot \partial z^2/\partial a^1 \cdot \partial a^1/\partial z^1 \cdot \partial z^1/\partial W^1$

Delta form: $\delta^3 = \partial L/\partial \hat{y}$, $\delta^2 = (W^{3T}\delta^3) \odot f'(z^2)$, $\delta^1 = (W^{2T}\delta^2) \odot f'(z^1)$, and $\partial L/\partial W^l = \delta^l (a^{l-1})^T$. Intermediate results are reused, which makes backpropagation efficient.

c) Updating weights

With gradients available for all layers, each parameter is updated simultaneously: $W^l \leftarrow W^l - \eta\, \partial L/\partial W^l$ and $b^l \leftarrow b^l - \eta\, \delta^l$. The next forward pass then yields a lower loss.

Task 6 – Activation Functions and Vanishing/Exploding Gradients (3 Marks)

a) Vanishing gradients

Backpropagation multiplies many derivative terms f′(z). Sigmoid has a maximum derivative of 0.25 and tanh of 1. For saturated inputs both derivatives approach 0. Across many layers the product shrinks exponentially (for example $0.25^{10} \approx 10^{-6}$), so early layers receive almost no gradient and learn very slowly. (Large weights can cause the opposite: exploding gradients.)

b) Comparison

Feature	**Sigmoid**	**Tanh**	**ReLU**
Formula	$1/(1+e^{-z})$	$(e^{z}-e^{-z})/(e^{z}+e^{-z})$	$\max(0, z)$
Range	(0, 1)	(−1, 1)	$[0, \infty)$
Zero-centred	No	Yes	No
Max derivative	0.25	1	1 (for $z > 0$)
Saturation	Both sides	Both sides	Only for $z < 0$
Vanishing gradient	Severe	Moderate	Largely avoided
Cost	Expensive (exp)	Expensive (exp)	Very cheap
Issue	Slow, not centred	Still saturates	Dying ReLU

c) Impact on deep training

ReLU (and variants such as Leaky ReLU, ELU) keeps gradients at 1 for active units, giving faster convergence and better learning in early layers. Sigmoid and tanh in deep hidden layers cause slow or stalled training. Sigmoid is best kept for binary output layers. Pairing ReLU with He initialization, batch normalization and residual connections further stabilizes gradient flow. Gradient clipping addresses exploding gradients.

Task 7 – Back-Propagation and Optimization Strategy (4 Marks)

a) Learning rate and optimization

η controls the step size. A too-small η gives slow, steady loss decrease (Experiment 1). A too-large η gives fast initial decrease but oscillation around or past the minimum (Experiment 2).

b) One iteration

1. Forward pass: compute predictions and loss on a batch.
2. Backpropagation: compute gradients of the loss for all weights using the chain rule.
3. Gradient descent: update the weights, $\theta \leftarrow \theta - \eta \nabla L$.

Backpropagation supplies the gradients; gradient descent uses them to change the parameters.

c) Improvements

Experiment	Problem	Suggested changes
1 – slow decrease	η too small	Increase η; use momentum or Adam; use batch normalization; proper (He) initialization; normalize inputs
2 – fast but fluctuating	η too large / noisy gradients	Reduce η; use a learning-rate decay schedule; increase batch size; use momentum or Adam; gradient clipping

d) Why monitor the loss curve

The curve shows convergence speed, instability (oscillation), divergence, plateaus and, when training and validation curves are compared, overfitting or underfitting. It guides decisions on learning rate, early stopping and regularization.

Task 8 – Regularization and Overfitting Control (4 Marks)

a) Problem and evidence

The likely problem is **overfitting** (high variance). Training accuracy of 99% versus validation accuracy of 78% is a 21-point gap. The model memorized training data, including noise, and does not generalize.

b) L1 and L2

- **L1 (Lasso)** adds $\lambda \Sigma |w_i|$ to the loss. It pushes many weights exactly to zero, giving sparse models and implicit feature selection.
- **L2 (Ridge / weight decay)** adds $\lambda \Sigma w_i^2$ to the loss. It shrinks all weights toward small values without making them exactly zero, giving smoother functions.

Both penalize large weights, thereby limiting effective model complexity.

c) L2-regularized loss

$J(w) = (1/N) \Sigma (y_i - \hat{y}_i)^2 + \lambda \Sigma w_j^2$

λ (the regularization parameter) sets the trade-off between fitting the data and keeping weights small. $\lambda = 0$ means no regularization. Small λ risks overfitting. Large λ causes underfitting. It is tuned on the validation set. The update becomes $w \leftarrow w(1 - 2\eta\lambda) - \eta \, \partial L/\partial w$, hence "weight decay".

d) Dropout

During training each hidden neuron is randomly switched off with probability p (e.g., 0.3–0.5). This prevents neurons from co-adapting, forces redundant, robust features, and acts like averaging an ensemble of many thinned networks. At inference all neurons are used, with outputs scaled accordingly (inverted dropout). It reduces the training/validation gap.

Task 9 – Convolution Operation and Feature Extraction (3 Marks)

a) Why convolutions suit images

- Local connectivity: each neuron looks at a small patch, capturing local patterns (spots, edges, texture).
- Parameter sharing: the same filter slides over the whole image, so far fewer parameters than a fully connected layer.
- Translation invariance: a lesion is detected wherever it appears on the leaf.
- Spatial structure (neighbouring pixels) is preserved, unlike flattening.
- Hierarchical features: edges → textures → disease patterns.

b) Example: vertical edge detection

Image (5×5), bright left region, dark right region:

9	9	9	0	0
9	9	9	0	0
9	9	9	0	0
9	9	9	0	0
9	9	9	0	0

Filter (3×3): rows [1, 0, −1], [1, 0, −1], [1, 0, −1]. Stride 1, no padding.

Window at columns 0–2 (all 9s): $3\times(9\times1 + 9\times0 + 9\times(-1)) = 0$. Window at columns 1–3 (9, 9, 0): $3\times(9 + 0 - 0) = 27$. Window at columns 2–4 (9, 0, 0): $3\times(9) = 27$.

Feature map (3×3): every row is [0, 27, 27].

Interpretation: 0 means a uniform region with no edge. High values (27) mark the boundary where brightness changes, i.e., an edge such as a lesion boundary.

Output size = $(n + 2p - f)/s + 1 = (5 + 0 - 3)/1 + 1 = 3$.

c) Filters, stride, padding

- **Filters (kernels)**: small learnable weight matrices; each detects one pattern (edge, spot, texture). Using k filters gives k feature maps.
- **Stride**: step size of the filter. Larger stride gives a smaller output and less computation but may lose detail.
- **Padding**: adding border pixels (usually zeros). "Same" padding preserves size and keeps border information; "valid" (none) shrinks the output.

Task 10 – CNN Architecture and Pooling (4 Marks)

a) Components

Component	**Function**
Input image	Raw leaf image, e.g., 224×224×3
Convolution	Filters extract local features (edges, spots, texture) into

Component	Function
	feature maps
ReLU	Adds non-linearity, avoids vanishing gradients
Pooling	Downsamples feature maps, reduces computation, adds small translation invariance
2nd Conv + ReLU + Pool	Combine low-level features into more complex ones
Fully connected layer	Flattens features and combines them for classification
Output	Softmax (healthy vs diseased) or sigmoid for binary

b) How pooling reduces dimensions

Pooling slides a window (e.g., 2×2, stride 2) and keeps one value per window (max or average). Output size = (n − f)/s + 1. Example: 4×4 → 2×2; 224×224 → 112×112, so each spatial dimension is halved and the number of values is reduced by 75%. Channel count is unchanged.

Example max pool of [[1, 3], [2, 4]] → 4.

Worked dimension example: 224×224×3 → Conv (3×3, 32 filters, same) 224×224×32 → Pool 112×112×32 → Conv (3×3, 64 filters, same) 112×112×64 → Pool 56×56×64 → Flatten 200,704 → FC → Output.

c) Effect of filter size, stride, number of filters

- Filter size: larger filters see more context but cost more parameters and compute and give smaller outputs (without padding). Small filters (3×3) are efficient and can be stacked.
- Stride: larger stride reduces the feature-map size and detail. Stride 1 keeps detail.
- Number of filters: sets the depth (channels) of the feature map. More filters mean more features learned but more parameters and memory.

d) Hierarchical learning

Early layers learn simple features (edges, colours, gradients). Middle layers combine them into textures and shapes (spots, vein patterns). Deeper layers, which have a larger receptive field, learn complex, disease-specific patterns (lesion types, leaf-level structures). The fully connected layer uses these for the final classification.

Task 11 – Comparative Evaluation of Feed-Forward and CNN Models (4 Marks)

a) Model A suits structured numerical data

Weather and soil data are low-dimensional tabular vectors with no spatial arrangement. Each feature is meaningful by itself, and a dense network can learn non-linear interactions (e.g., rainfall × fertilizer). It is simple, fast and needs fewer parameters.

b) Model B suits images

Images have spatial structure. Convolution exploits local patterns, shares parameters, and detects disease features anywhere in the leaf. This also avoids the huge parameter count of fully connected layers on pixels.

c) Comparison

Aspect	Model A (FFN)	Model B (CNN)

Aspect	Model A (FFN)	Model B (CNN)
Input representation	Flat vector of 4 numeric features	3D tensor (H × W × channels)
Feature extraction	Manual features supplied; learned via dense mixing	Automatic, hierarchical via filters
Parameter organization	Full weight matrices, each input connected to each neuron	Small shared filters, sparse connectivity
Computation	Low (tiny parameter count)	High; needs GPU and more memory
Spatial awareness	None	Preserves spatial relationships

d) Recommendation

- Yield prediction: **Model A** (feed-forward, regression). Data is tabular and low-dimensional, so it is cheap, accurate and easy to train.
- Leaf disease classification: **Model B** (CNN). It learns visual features automatically and generalizes far better than a dense network on pixels.

Task 12 – Design Challenge: Optimize and Redesign the System (8 Marks)

A. Impact Analysis (2 Marks)

a) Effect of larger data and images

- 500,000 records (vs 10,000, 50× more): more compute per epoch, so mini-batch training (e.g., 256–1024) and efficient data pipelines are needed. A larger dataset supports a somewhat bigger model and reduces overfitting.
- 100,000 leaf images add high-dimensional input (e.g., 224×224×3 = 150,528 values per image), large memory and GPU requirements, longer training, and the need for spatial feature extraction and data augmentation. The system must now be multi-modal (tabular + image) and handle two different tasks.

b) Limits of feed-forward networks on images

- Flattening destroys spatial structure.
- Huge parameter count: 150,528 inputs × 512 neurons ≈ 77 million weights in only the first layer, causing overfitting and heavy compute.
- No translation invariance: a lesion in a different position looks like a new pattern.
- No parameter sharing and poor generalization.

B. Evaluation of the Existing Design (2 Marks)

a) Evaluation

High training accuracy with lower validation performance indicates overfitting (high variance). Long training time shows computational inefficiency (likely full-batch or unoptimized training). The model is not generalizing and cannot handle images.

b) Is more depth and neurons the best fix?

No. More capacity usually worsens overfitting, increases training time, and can aggravate vanishing gradients. Better remedies are regularization (L2, dropout), more data, early stopping, batch normalization, better

optimizers and an architecture matched to the data type (CNN for images). Capacity should only grow if validation error shows underfitting.

C. Adaptation or Redesign (2 Marks)

a) Proposed architectures

- **Numerical yield model (FFN):** Input(4) → Dense(64, ReLU) + BatchNorm → Dropout(0.3) → Dense(32, ReLU) → Dropout(0.2) → Dense(1, linear). Loss: MSE or Huber.
- **Leaf-image model (CNN):** Input(224×224×3) → [Conv 3×3(32) + BN + ReLU → MaxPool] → [Conv 3×3(64) + BN + ReLU → MaxPool] → [Conv 3×3(128) + BN + ReLU → MaxPool] → Global Average Pooling → Dropout(0.5) → Dense(1, sigmoid) (or softmax for multiple diseases). Loss: binary cross-entropy. A pretrained backbone (transfer learning) is recommended given the data volume.

b) Optimization and regularization

- Optimizer: **Adam** (lr = 1e-3) as default for fast convergence, with **SGD + momentum** (0.9) as a strong choice for fine-tuning CNNs.
- Learning-rate scheduling: warm-up, then cosine decay or ReduceLROnPlateau.
- Regularization: L2 weight decay (1e-4 to 1e-5), dropout (0.2–0.5), early stopping, data augmentation.
- Batch normalization to stabilize and speed training.
- Activations: ReLU/Leaky ReLU in hidden layers, linear output for regression, sigmoid/softmax for classification. He initialization.

D. Evidence-Based Defence (2 Marks)

a) Experimental design

Split the data into train/validation/test (e.g., 70/15/15, stratified for classes). Train the original system and the redesigned system under the same conditions and compare, across several random seeds:

Metric	**Yield model**	**Leaf model**
Training and validation loss curves	MSE	Cross-entropy
Performance	MAE, RMSE, R^2	Accuracy, Precision, Recall, F1, ROC-AUC, confusion matrix
Generalization gap	Train − val loss	Train − val accuracy
Training time	Time per epoch and total to convergence	same

Final results are reported on the untouched test set. Hyperparameters are tuned only on the validation set.

b) Decision

Redesign the system. The original shows overfitting and long training time, and cannot process images. Expected evidence: lower validation loss, a smaller train–validation gap, higher F1 on diseased leaves, and shorter training time (mini-batch + Adam + BN). The FFN is retained but **modified** for tabular yield prediction, while a new CNN is added for images.

Additional Design Considerations

Optimizer comparison

Optimizer	Idea	Pros	Cons
SGD	Update using one sample	Cheap per step, can escape local minima	Very noisy, slow
Mini-batch SGD	Update using a batch	Stable, GPU-efficient, standard	Needs batch-size and lr tuning
Momentum	Accumulates past gradients	Faster, damps oscillations	Extra hyperparameter
RMSProp	Per-parameter lr scaled by running average of squared gradients	Good for non-stationary problems	Needs lr tuning
Adam	Momentum + RMSProp with bias correction	Fast, robust defaults	Can generalize slightly worse than SGD

Learning-rate schedule: warm-up for 5 epochs (1e-4 → 1e-3), then cosine annealing to about 1e-6 over the remaining epochs, or ReduceLROnPlateau (factor 0.1, patience 5) with early stopping.

Label-preserving augmentation: random horizontal/vertical flips, rotation (±30°), small zoom/crop (0.8–1.0), brightness and contrast jitter (±20%), slight colour/saturation shifts, small translations, mild Gaussian noise or blur. Avoid heavy colour changes or aggressive crops that remove lesions or alter the disease appearance (e.g., hue shifts that mimic discolouration). Apply augmentation to the training set only.

CNN comparison for the given image size and budget

Model	Depth / params	Input size	Strengths	Limits
LeNet	~5 layers, ~60K	32×32	Very light, simple	Too small for detailed leaf textures
AlexNet	8 layers, ~60M	227×227	Landmark deep CNN, ReLU + dropout	Heavy, many parameters, outdated
ResNet (e.g., ResNet-50)	50 layers, ~25M	224×224	Skip connections solve vanishing gradients, strong transfer learning	Higher compute
EfficientNet (B0)	~5M	224×224	Best accuracy per FLOP via compound scaling	Slower to train from scratch, sensitive to settings

Recommendation: fine-tune a pretrained **EfficientNet-B0** (or ResNet-50 if more GPU is available) at 224×224 for the best accuracy-to-compute balance on 100,000 images. A small custom CNN is a good baseline.